

Design Optimization of Computing Systems

Assignment 1: Understanding, Tuning, Extending, and Benchmarking Lua's Garbage Collector

Deadline: 11:59 PM, 30th August, 2026

Maximum Marks: 200

Team Size: 2-4

Lua is a lightweight, high-level programming language designed for embedding within applications. It is known for its simplicity, efficiency, and small footprint, making it ideal for use in embedded systems, game development, and scripting. Lua features a clean syntax, dynamic typing, and automatic memory management through garbage collection. It's highly extensible, allowing integration with C/C++ libraries, and supports cross-platform development. Lua's versatility and ease of embedding have made it a popular choice for adding scripting capabilities to a wide range of software applications. Lua's interpreter is written in C.

In this assignment you will play with Lua's Garbage Collector end to end, you will understand how it works, patch it to make one of its internal parameters tunable, extend the collector itself with a static escape analyzer and a stack-allocation fast path, and finally benchmark everything you built, the stock collector, your tunable parameter, and your extension, together, in one consolidated comparison.

This assignment is supposed to give you an idea regarding:

- how to read and understand the source code of any program in Lua.
- how to modify a real, production-grade collector's source and add a tunable parameter to it.
- how to design and implement a compiler-level static analysis and wire its result into the runtime's allocator.
- how to benchmark any Lua program, isolate the cost and the benefit of a change, and identify potential problem areas.

The assignment is presented in 4 parts as follows, all of which build on each other and use the same Lua build and the same testbench throughout:

- **Part A:** you read Lua's real GC source and use Valgrind/callgrind to understand how its three configurations (full, incremental, and generational) actually behave, function call by function call.
- **Part B:** you patch that same collector's source directly to turn one of its internal constants, the sweep granularity, into a tunable parameter.
- **Part C:** you extend the compiler and the collector with a static escape analyzer and a frame-arena allocator so that some table allocations can bypass the GC heap safely.
- **Part D:** you benchmark everything from Parts A, B, and C together, the stock GC configurations, the tunable sweep-step parameter, and the escape-analysis promotion, and bring every scenario into a single consolidated comparison.

Materials provided with this assignment: `lua5.5-student-src.zip` (Lua 5.5.0, a frozen snapshot) and `testbench.lua` (the workload used throughout all four parts, provided in Appendix A). Build from the provided snapshot; do not clone or download Lua from lua.org or GitHub yourselves.

Part A: Understanding Lua's GC (50 marks)

In this part, you will understand how Lua's GC works by reading its source code and using Valgrind to see how it actually runs. To begin with, unzip the provided `lua5.5-student-src.zip`

and build it. Verify with `./lua -v` (expect “Lua 5.5.0”).

After you have built it, proceed as follows:

- You are given a sample testbench (see Appendix A). You will be using this testbench throughout Parts A, B, C, and D.
- Write 3 C files named `fullgc.c`, `incrementalgc.c` and `generationalgc.c` which will load the testbench file and run it under different GC configurations, i.e. full GC, incremental GC and generational GC, using `lua_gc()`’s `LUA_GCCOLLECT`, `LUA_GCINC`, and `LUA_GCGEN` options respectively.
- Note: Lua being written in C allows you to load and run Lua files from C. You are not allowed to use `fork` and `exec` system calls.
- Run `valgrind` on the three executables, then parse the output using tools like `gprof2dot` or `kcachegrind` to generate call graphs.
- Use the call graphs to read the source code and understand which internal functions each GC configuration actually calls, how often, and in what order.

Deliverable: Submit the three C files along with a report on how Lua’s GC works in each configuration, based on what the call graphs show you, in not more than 5 pages.

Part B: Sweep Granularity, Making It Tunable (40 marks)

Part A showed you Lua’s collector from the outside. In Part B you will change the Lua’ GC source directly, from your driver to control this; the change must live inside `lgc.c` itself, at file scope.

`GCSWEEPMAX` is currently a hardcoded `#define` in `lgc.c` controlling how many objects `sweepstep()` frees per incremental step:

- Turn `GCSWEEPMAX` into a static variable, settable through a small function you add, `lua_setsweepstep(lua_State *L, int n)`, declared with `LUA_API` in `lua.h` next to `lua_gc`, and defined in `lgc.c`.
- Rebuild the whole interpreter after editing.

Deliverable: your modified `lgc.c` / `lua.h`, and the diff against the originals, and a short report in a `README.md` file summarizing the design and changes you have made.

Part C: Static Escape Analysis for Stack Promotion (50 marks)

Parts A and B treat Lua’s GC as a fixed cost you tune around; you can pick a configuration (full / incremental / generational) and you can tune when the sweeper does work (`GCSWEEPMAX`), but every table allocation still ends up on the GC heap and still eventually gets swept. In Part C you will explore a different question, *can some allocations avoid the GC heap entirely?*

This is the idea behind Rust’s ownership model (prove a value’s lifetime statically, skip the tracer) but adapted to keep Lua’s defining property, the GC is never removed, only bypassed *when provably safe*. Every allocation the analysis can’t fully reason about simply falls back to the existing collector from Parts A and B, unchanged. Nothing about program semantics changes; only where a table’s storage physically lives changes. In this assignment you will have to perform the following tasks.

Task

C.1: Escape Analysis (25 marks)

Implement a static escape analyzer for local table allocations (`local t = {...}`), hooked into the compiler pass in `lparser.c` (you may also implement it as a separate pre-pass over the AST/bytecode if you prefer, as long as it runs at compile time, not runtime).

A table local is **non-escaping** (promotable) unless the analyzer proves one of the following at a specific syntactic site:

1. **Return-escape**, the table (or anything holding a reference to it) is part of a **return** expression.
2. **Capture-escape**, the table is referenced inside any nested closure defined in its scope. (Conservative: treat *any* capture as escaping, you are not required to analyze whether the closure itself escapes.)
3. **Store-escape**, the table is stored as a key or value into another table that is itself escaping (globals are always escaping).
4. **Call-escape**, the table is passed as an argument to any function call, *unless* the callee is on a small whitelist of analyzer-known-safe builtins that provably don't retain arguments beyond the call (e.g. `type`, `print`, `#`, iteration-only use in `pairs`/`ipairs`).

Rule 4 is the conservative catch-all: an unrecognized callee is guilty-by-default. This is what makes false positives (wrongly promoting an escaping table) structurally impossible, you only ever fail toward the GC, never away from correctness.

You only need a fixed point **within a single function body** (a worklist over that function's table-locals, typically converging in 1–2 passes), no whole-program / interprocedural analysis is required.

Deliverable: your modified `lparser.c` (or added source files), a short design note on how the four rules are checked, and the diff against the originals.

C.2: Frame-Arena Promotion (25 marks)

For every table local proven non-escaping, allocate it from a per-call-frame bump arena instead of via `luaC_newobj` (i.e., it never gets `GCObject` linkage and is invisible to `sweepgc`). The arena is popped in bulk when the frame returns.

- Same `Table` struct, same field-access code paths, only the allocator and lifetime differ. Do not add a per-access “is this heap or stack” branch to the general table-access path; that would add overhead everywhere to save it in one place.
- Bail to the normal GC allocator for any table your C.1 analysis didn't clear or that doesn't fit the frame arena's assumptions (e.g., a table escapes the frame via a coroutine yield).

Deliverable: your modified `lgc.c` / `lmem.c` / `lobject.c` or any other files with diff against originals, and a short note with a `README.md` file showcasing the changes you have made.

Notes on Scope

- You are **not** required to implement scalar replacement (splitting a table into individual stack slots), the frame-arena approach in C.2 is sufficient and is the intended scope.
- You are **not** required to handle coroutines beyond detecting that a table crosses a `coroutine.yield` boundary and treating that as an automatic escape (rule 4).

Part D: Benchmarking Everything Together (50 marks)

Parts A, B, and C gave you a working understanding, a working tunable parameter, and a working compiler extension. Part D is where you actually measure all three, and put every scenario side by side so that conclusions become visible that would not be if you benchmarked each part in isolation.

D.1: Benchmarking the Stock GC Configurations (12 marks)

Using the three C files from Part A, benchmark Lua's stock full, incremental, and generational configurations:

- Use the callgrind outputs or call graphs to find the percentage of instructions consumed by GC for the different GC configurations. Report the results and explain the variation.
- Re-run the previous test under the following parameter sets (on the same testbench): $m = 100, n = 100$; $m = 500, n = 100$; $m = 5000, n = 100$. Report the results and explain any trend that you see.
- Run perf analysis on the same testbench with the original parameters and report Branch Misses, Page Faults, Cache Misses, and Instructions Per Cycle for the different GC configurations and a configuration without GC (`lua_gc(L, LUA_GCSTOP)`). For each metric, explain why it is relevant for this scenario, and explain the variation across configurations.

Deliverable: results, explanations, and answers for all three sub-parts above, plus call graph images and callgrind outputs for each parameter set.

D.2: Benchmarking the Sweep-Step Tuning (12 marks)

Using the interpreter you patched in Part B, benchmark the effect of your new tunable parameter:

- Run the testbench from Appendix A at ≥ 4 values of your sweep-step setting, e.g. 5, 20 (the default), 50, and 200.
- For each value, report (a) total wall-clock / instruction-count cost of the full run, and (b) the distribution of individual `GCSwpallgc` step costs from callgrind, since that is what granularity actually changes.

Deliverable: the benchmarking results for every tested sweep-step value.

D.3: Benchmarking the Escape Analyzer and Its Promotion Win (16 marks)

Report these as two separate numbers, not one blended before/after figure:

- **Cost of the analysis itself:** compile-time overhead with vs. without the C.1 pass enabled, on the testbench from Appendix A.
- **Benefit to the collector:** using the same callgrind methodology as D.1, report the reduction (if any) in objects reaching `sweepgc`, `GCSwpallgc` step counts, and instructions consumed by GC, with C.1+C.2 enabled vs. stock, across the same (m, n) parameter sets used in D.1.

Deliverable: the two separate cost/benefit numbers, plus an explanation of any workloads where promotion helps little or not at all.

Submission Guidelines

Zip all the files into a single zip file named:

Asgn1_<RollNumber1>_<RollNumber2>[_<RollNumber3>].zip

and upload it on Teams. Only one submission per team.

Evaluation Guidelines

The total marks for this assignment = 200, distributed equally (50 marks each) across the four parts.

Part	Item	Marks
Part A: Understanding Lua's GC	C files (3×4)	12
	Report on each GC configuration (13+13+12)	38
Part A Total		50
Part B: Sweep Granularity	Implementation of tunable sweep step	15
	README Description	15
	Diff correctness	10
Part B Total		40
Part C: Static Escape	C.1: Escape analyzer implementation	20
	C.1: Design note	10
	C.2: Frame-arena allocator + wiring	21
	C.2: Design note	9
Part C Total		60
Part D: Benchmarking Everything	D.1: Stock GC benchmarking	17
	D.2: Sweep-step benchmarking	17
	D.3: Escape-analyzer cost & GC benefit	16
Part D Total		50
Grand Total		200

Appendix A: Test Bench Code

Following is the testbench code. It is commented well for you to understand what it is doing, and it is also provided as `testbench.lua` alongside this document.

```
— This is a testbench. This file is to be used to benchmark Lua's GC.
function test()
    local survivors = {}

    — Parameters.
    — m, n = m * n total allocations.
```

```

— survive_every = 1 out of every N objects is kept alive permanently;
— the other N-1 are garbage the instant the iteration ends.
local m = 1000
local n = 100
local survive_every = 100

local count = 0
for i = 1, m do
  for j = 1, n do
    count = count + 1

    local obj = {}
    obj[1] = "payload"
    obj[2] = {i, j}

    if count % survive_every == 0 then
      — Promoted: referenced from ‘survivors‘ for the rest
      — of the run, so it outlives many collection cycles.
      survivors[#survivors + 1] = obj
    end
  end
end

return survivors
end

test()

```